

Lab 1 and 2

India Air Quality & Crop Yield — EDA Lab

Data Preprocessing, Visualisation & Exploration

Combined Lab Sheet

June is globally recognised as [Environment Awareness Month](#), with World Environment Day celebrated on June 5 to encourage action to protect our planet and promote sustainable living.

In connection with this global initiative, this lab exercise focuses on exploring environmental and agricultural data through the lens of Machine Learning and Data Analytics.

Aim

To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.

Objectives

- Independently assess data quality and decide on appropriate treatment strategies
- Select and justify suitable visualisation techniques for a given analytical problem
- Detect anomalies, patterns, and relationships in environmental data
- Derive and communicate meaningful inferences to both technical and non-technical audiences
- Develop critical thinking around data limitations and confounding factors
- Dataset provided

Two CSV files are provided:  dataset

[city_aqi.csv](#) contains city-level air quality readings (City, State, Date, AQI, PM2.5, PM10, NO2, CO) from 2015–2023. [crop_yield.csv](#) contains state-level crop production records (State, Year, Crop, Season, Area, Production) for the same period.

The files are raw and unverified. They may contain missing values, inconsistent entries, duplicate records, and extreme anomalies. It is your responsibility as an analyst to discover and address all issues before drawing any conclusions.

Each task below presents a real-world problem. You must decide what to do, how to do it, and why — then document your reasoning in a markdown cell alongside your code. There are no prescribed methods unless stated.

Lab 1

Data preprocessing and visualisation

10 marks

Task 1

A junior analyst hands you two CSV files and says — "I have no idea what's in these. We need to use them to build a machine learning model next week." Before any analysis can begin, you need to understand what you are working with.

Investigate both files thoroughly. Produce a structured summary that gives a complete first-impression picture of the data — its size, its contents, and where it might be problematic. Decide what information a data scientist would need before trusting this data, and make sure your summary covers all of it.

A structured data profile for each file in a markdown cell

At least one written observation about what concerns you after the inspection

Think: What does a data scientist need to know about a dataset before using it? What functions help you find that?

2 marks Completeness of profile + written concern

Task 2 The data has holes — and not all holes are equal

After the inspection, it is clear that several columns have missing values. A colleague suggests simply deleting all rows with any null value. Another suggests filling everything with the mean. You are not sure either approach is right.

Devise your own missing value treatment strategy. For each column with missing data, decide whether to drop it, drop affected rows, or impute — and explain why that choice is appropriate for that specific column. Apply your strategy and verify it worked.

A markdown cell that justifies each decision column by column

Before and after null counts as evidence the treatment worked

Think: Does the volume of missing data matter? Does the column type matter? What does imputing with mean vs median assume about the data? 2 marks

Task 3 The two files disagree on how to spell "Tamil Nadu"

You need to combine both files later in the lab using the State column as the common key. But when you look closely, the same state appears with different spellings, extra spaces, or old names across the two files. There are also rows that appear more than once for no clear reason.

Make both files agree on state names and remove any redundant records. Document every inconsistency you found and how you resolved it. Show that the files are now ready to be merged reliably.

A list of all inconsistencies found and the fix applied for each

Record counts before and after to prove duplicates were removed

Think: What could go wrong in a merge if state names don't match exactly? How would you systematically find all variants of a state name? 2 marks

Task 4

Where do most cities actually sit on the AQI scale?

The pollution control board wants to know — are most Indian cities moderately polluted, or is the problem concentrated in just a few places? They also want to know whether the average AQI is a fair number to report publicly, or whether extreme cities are pulling it up unfairly.

Investigate the shape of the AQI distribution. Decide which visualisation(s) best answer both questions the board is asking, justify your choice, produce the plot(s), and record two specific observations that directly address their concerns.

Your chosen plot(s) with fully labelled axes and a descriptive title

A markdown cell with your justification for the chosen plot type and two observations

Think: Which plot shows where values cluster? Which one reveals extreme values? Do you need one plot or two to answer both questions? 2 marks

Task 5

Something is making the average AQI look worse than it is

A data quality report flags that a few AQI readings in the dataset are implausibly high — values that no monitoring station should realistically record. If left in, these values will distort every statistic and model built on this data. The team lead asks you to "handle the extreme values properly" but does not tell you how.

Identify whether extreme values exist, quantify them, decide on an appropriate treatment method, apply it, and demonstrate that the data is now cleaner. Justify every decision you make.

The method you chose to detect extremes and why

The count of values affected and the treatment applied

A visual comparison of the data before and after treatment

Think: Should extreme values always be deleted? What are the alternatives? How do you prove your treatment actually worked? 2 marks

Lab 2

Data exploration and inferences

10 marks

Task 6: Is India's air getting better or worse over time?

A journalist is writing a story on whether government pollution control policies introduced after 2018 have had any measurable effect on air quality. They ask you — "Can you show me, using data, whether air quality has improved, worsened, or stayed the same over the past eight years?"

Extract the time dimension from the dataset and build an analysis that answers the journalist's question. Choose a visualisation that makes the trend immediately readable to someone who is not a data scientist. Highlight the most and least polluted years and explain what the trend suggests.

Your plot with the trend clearly visible and key years highlighted
A markdown response to the journalist's question — one paragraph, plain language

Think: What needs to happen to the Date column before you can group by year? Which plot type communicates a trend over time most clearly? 2 marks

Task 7 Farmers say the air is worse exactly when they harvest — is that true?

An agricultural NGO claims that air quality is consistently worse during the October–December harvest season, when crop residue burning is widespread. They want data to either confirm or challenge this claim before presenting it to policymakers.

Investigate whether AQI follows a seasonal pattern across the year. Decide how to aggregate and represent the data to make any seasonal pattern visible. Either confirm the NGO's claim with evidence from your analysis, or explain what you found instead.

A visualisation that clearly shows how AQI varies across months or seasons
A markdown cell that directly responds to the NGO's claim with data evidence

Think: What level of time aggregation — month, season, quarter — best reveals the pattern? How do you extract that from a date column? 2 marks

Task 8: Can the two datasets talk to each other?

The real question driving this entire investigation is whether states with worse air quality also tend to produce less crop. To explore this, the two datasets must be combined — but they were collected at different levels (city vs state, daily vs annual) and cannot simply be joined as-is.

Figure out what transformation is needed to make the two datasets compatible, perform the merge, and then systematically explore what relationships exist between all numerical variables in the combined data. Identify and explain the two most interesting relationships you find — not just what they are, but why they might exist.

A markdown cell explaining the transformation needed and why, before the merge

A visualisation of relationships across all numerical features

Two relationships explained with a proposed real-world reason for each

Think: If one file has city-day rows and the other has state-year rows, what needs to happen before they can be joined? What does a correlation matrix actually tell you — and what doesn't it tell you? 3 marks

Task 9 : The minister needs to act — what do you tell her?

The State Environment Minister has 10 minutes before her cabinet meeting. She has never opened a Jupyter notebook. Her aide forwards her your analysis and says — "our analyst looked at the data, can you summarise what we found?" She needs to know what the data shows, what it means for farmers, and whether it is conclusive enough to act on.

Write a clear, honest briefing addressed directly to the minister. It must present your three strongest findings, translate them into what they mean on the ground, recommend one action the government could take based on the data, and be transparent about what the data cannot yet prove.

150–200 words in a markdown cell, addressed to the minister. Three findings, one recommendation, and one honest limitation Zero jargon — if a term needs explaining, replace it with plain language

Think: Which of your findings actually matters to a farmer or a policymaker? What is the difference between correlation and proof? Would the minister act on what you found? 3 marks

Optional — advanced task

Build the case — or break it

Attempt only after completing Tasks 1–9. The central hypothesis of this entire investigation is: states with higher pollution have lower crop yields. Your job here is to either build the strongest possible case for this hypothesis — or find evidence that challenges it.

Task A: The two extremes — do they tell the same story?

If the hypothesis is true, you would expect the most polluted states and the least polluted states to show a clear difference in crop output. But the data might surprise you.

Identify the states that represent the two extremes of pollution. Compare their agricultural output using a visualisation of your choice. Analyse what you see — does the pattern support the hypothesis, contradict it, or is it more complicated than that?

Task B: Put a number on the relationship

A research team wants to know not just whether a relationship exists between AQI and crop production, but how strong it is and in which direction. They will use your output to decide whether to fund a full causal study.

Quantify the relationship between average state AQI and average crop production. Choose a method that both measures the strength and makes the pattern visible in a single output. Interpret your result for the research team — be honest about what it proves and what it does not.

Think: What does a correlation value of 0.2 vs 0.8 mean in practice? Does a strong correlation mean pollution is causing yield loss?

Task C: One plot to rule them all

You have produced many visualisations across this lab. A data journalism outlet wants to publish just one chart from your work to accompany a story on India's air-agriculture crisis.

Choose the single visualisation from your entire lab that you believe most powerfully tells the story of this dataset. Write a 3–4 sentence caption for it — not a description of what the chart shows, but an explanation of why it matters and what it reveals that no other chart in your analysis does.

Caption reveals insight, not just description

Across all optional tasks, your writing must acknowledge that correlation is not causation and name at least one factor — rainfall, irrigation, soil type, economic conditions — that this dataset cannot account for but which could explain any relationship observed.

Deliverables

Single Jupyter Notebook (.ipynb) — tasks numbered, code commented, every decision explained in a markdown cell

Minimum 5 visualisations of your own choosing — axes labelled, titles present

Markdown reasoning cells for every task — not optional, part of the marks

Lab 3

Lab Exercise 3 :

Student Survey: Simple Linear Regression using Scikit Learn, OLS, and Parameter Saving

Aim

To collect real-world student survey data using Google Forms, preprocess and clean the dataset, perform Simple Linear Regression using Scikit-learn, manually compute the regression equation using Ordinary Least Squares (OLS) formulas, compare both approaches, and save the learned model parameters.

Instructions

1. During the first 15 minutes of the lab session, fill out the Google Form shared, link :

MCA-B

<https://docs.google.com/forms/d/e/1FAIpQLSdR13N42Eh2Lz7U7QYkw9bp-G03puOGKROVQz4fzUDGdF1g8A/viewform?usp=publish-editor>

MCA-A

<https://docs.google.com/forms/d/1cMs0pEILKDkqJEUt2TYRdZg3rJz7-aTelD6uT3EW8VA/edit?ts=6a2ecba6>

2. Ensure that responses are collected from your classmates.
3. Export the responses into CSV format.
4. Use the collected dataset for all further analysis

Suggested dataset name:

student_survey.csv

Problem Statement

Using the dataset collected from the class survey form, perform Simple Linear Regression analysis to determine whether:

1. CIA percentage can predict GPA

2. Attendance percentage can predict GPA

You are required to perform regression in two different ways:

1. Using Scikit learn LinearRegression
2. Manually using the mathematical equations of slope and intercept derived from Ordinary Least Squares (OLS)

Finally, compare the outputs obtained from both methods and save the learned model parameters.

Refer this for getting and understanding :

https://colab.research.google.com/drive/1No__uEMN0sq8y2oqGvRgL9FKMGAp7BxX#scrollTo=a7bfa6e8

Tasks to Perform

Part A: Data Collection and Preprocessing

Perform the following steps:

1. Load the dataset using Pandas
2. Display the first 5 rows
3. Check dataset dimensions
4. Identify missing values
5. Remove or handle null values
6. Convert required columns into numerical datatype
7. Remove duplicate records if present
8. Generate statistical summary
9. Select appropriate dependent and independent variables

Regression Experiments: Experiment 1

Independent Variable (X):

CIA Percentage

Dependent Variable (Y):

GPA

Experiment 2

Independent Variable (X):

Attendance Percentage

Dependent Variable (Y):

GPA

Part B: Simple Linear Regression using Scikit-learn

Perform the following:

1. Split the dataset into training and testing sets
2. Train the model using LinearRegression from Scikit learn
3. Obtain:
 - Slope
 - Intercept
4. Predict output values using the trained mod

Part C: Manual Computation using Ordinary Least Squares (OLS)

Using the same dataset, manually compute the regression line using the OLS formulas given below.

Slope equation m :

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Intercept equation b :

$$b = \bar{y} - m\bar{x}$$

Final regression equation:

$$\hat{y} = mx + b$$

Where:

- x_i = individual input value
- y_i = individual output value
- $\bar{x}$ = mean of x values
- $\bar{y}$ = mean of y values
- m = slope
- b = intercept
- $\hat{y}$ = predicted value

Refer :

Manual Regression Equation

Construct the regression equation manually using the computed slope and intercept values.

Comparison Task

Compare the following:

1. Predictions from Scikit learn
2. Predictions from manually computed OLS equation
3. Difference between both outputs

Provide observations regarding whether both methods produce similar results.

Parameter Saving Task

Save the parameters learned by the model using Pickle.

The following parameters must be saved:

1. Slope
2. Intercept

Suggested file name:

linear_regression_weights.pkl

Demonstrate:

1. Saving parameters into Pickle file
2. Loading the Pickle file
3. Using loaded parameters for prediction

Expected Output

Submit the following:

1. CSV dataset
2. Data preprocessing notebook
3. Scikit learn regression implementation
4. Manual OLS calculation(self study)
5. Comparison of both approaches
6. Pickle file containing saved parameters
7. Final observations and inference

Sample Viva Questions

1. What is Simple Linear Regression?
2. What is the role of slope and intercept?
3. What is Ordinary Least Squares?
4. Why do we square the errors in OLS?
5. Difference between dependent and independent variable
6. Why should data be cleaned before training?
7. Why are slope and intercept called model parameters?
8. Why do we save learned weights?

Lab 4

Lab 4: Regression and Classification Evaluation Metrics

Part 1: Comprehensive Study of K-Nearest Neighbours (KNN) Classification using Breast Cancer Dataset and Comparison with Regression Evaluation Metrics

Aim

To implement KNN classification on the Breast Cancer dataset and analyze model performance using train-test split, heuristic K selection, cross-validation, ROC-AUC, and classification metrics. Also, to compare classification metrics with regression metrics studied in Linear Regression (Lab 3).

Dataset

Breast Cancer Wisconsin (Diagnostic) Dataset

From UCI : <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Or From kaggle :

<https://www.kaggle.com/datasets/utkarshx27/breast-cancer-wisconsin-diagnostic-dataset>

Or from csv :

https://drive.google.com/file/d/1poP5CatfiZGaRbGCAKPGe_wOBCGspDjA/view?usp=drive_link

- 569 samples
- 30 numerical features
- Target:
 - 0 → Malignant
 - 1 → Benign

Problem Statement

A healthcare analytics team is developing a predictive model for early cancer detection. You are required to build a KNN classifier, optimize its performance using different validation techniques, and compare classification evaluation metrics with regression evaluation metrics from Lab 3.

Tasks

Task 1: Data Preparation

1. Load the dataset using sklearn.

2. Convert into DataFrame and explore structure.
3. Check missing values and duplicates.
4. Apply feature scaling using StandardScaler and justify its importance.

Task 2: Train-Test Split Analysis

1. Split the dataset into training and testing sets (80:20).
2. Repeat with 70:30 and 90:10 splits.
3. Compare performance variations across splits.
4. Analyze how dataset splitting affects model stability and generalization.

Task 3: KNN Model with Heuristic K Selection

3.1 Heuristic Method for K Selection

1. Compute initial K using heuristic rule ($K = \sqrt{n}$); where n = number of training samples
2. Use this value as a baseline K.

3.2 Model Training

1. Train KNN classifier using heuristic K value.
2. Experiment with nearby values of K ($K \pm 5$).
3. Plot accuracy vs K values.
4. Identify optimal K based on performance trends.

3.3 Distance Matrix and Decision Boundary Mapping

1. Explain any two distance metrics used in KNN: **Euclidean Distance** and **Manhattan Distance**. Also mention when each distance metric is suitable.
2. Plot the **decision boundary** of the KNN classifier for different values of K, such as $K = 1$, $K = 5$, $K = 10$, and $K = 20$. Analyze how the decision boundary changes as K increase

Task 4: Cross Validation

1. Apply K-Fold Cross Validation ($k = 5$ or 10).
2. Compute mean accuracy for different K values.
3. Compare cross-validation results with train-test split results.
4. Select best K based on both heuristic + validation results.

Task 5: Classification Evaluation

Evaluate final model using:

- Accuracy

- Precision
- Recall
- F1 Score
- Confusion Matrix
- ROC Curve and AUC Score

Task 6: Comparative Study with Regression (Lab 3 Integration)

Reference: Recall **Lab 3 (Linear Regression)** where you evaluated:

- MAE
- MSE
- RMSE
- R^2 Score

Tasks:

1. Compare regression evaluation metrics with classification metrics.
2. Explain differences between:
 - Error-based evaluation (Regression)
 - Decision-based evaluation (Classification)
3. Compare:
 - R^2 Score vs Accuracy
 - RMSE vs F1 Score
 - MAE vs Confusion Matrix
4. Discuss differences in evaluation logic for:
 - Continuous prediction tasks
 - Classification tasks

Inference Requirement (short and precise -don't generate)

Write a detailed inference covering:

- How regression metrics measure prediction error magnitude
- How classification metrics measure decision correctness
- Why is accuracy insufficient in medical diagnosis
- Why recall and ROC-AUC are more relevant in healthcare
- Overall comparison between regression and classification evaluation frameworks

Task 7: Analytical Questions

1. Why is KNN called a lazy learning algorithm?
2. Why is feature scaling required in KNN?

3. Explain heuristic K selection using $\sqrt{n}$ rule.
4. Why is cross-validation more reliable than a single train-test split?
5. How does K affect bias-variance trade-off?
6. Why is recall more important than accuracy in cancer prediction?
7. What is the limitation of very large K values?

Expected Outcome

- Understanding of KNN algorithm and distance-based learning
- Ability to tune K using heuristic + validation
- Knowledge of cross-validation
- Interpretation of ROC-AUC in medical classification
- Strong conceptual link between regression and classification evaluation metrics

Conclusion

Students should summarize:

- Optimal K value using $\sqrt{n}$ and validation
- Effect of train-test split variations
- Model performance based on evaluation metrics
- Key differences between regression and classification evaluation
- Insights from Lab 3 vs current lab

Lab 5

Lab Experiment: Linear Regression through Gradient Descent

Aim

To implement Linear Regression using the Gradient Descent optimization algorithm and evaluate its performance on a real world dataset.

Objectives

1. To understand the concept of Gradient Descent for optimizing a Linear Regression model.
2. To preprocess the dataset before model training.
3. To implement Linear Regression using Gradient Descent.
4. To analyze the convergence of the loss function during training.
5. To evaluate the model using standard regression metrics.

Dataset

Dataset Name: Student Performance Dataset

Source: UCI Machine Learning Repository

Dataset Link:

<https://archive.ics.uci.edu/dataset/320/student+performance>

Tasks to Perform

1. Download and load the Student Performance dataset.
2. Perform data preprocessing, including handling missing values (if any), encoding categorical variables, and feature scaling.
3. Select appropriate input features and the target variable.
4. Split the dataset into training and testing sets.
5. Implement Linear Regression using the Gradient Descent algorithm.
6. Experiment with different learning rates and observe their effect on model convergence.
7. Plot the loss (cost) versus the number of iterations.
8. Evaluate the trained model using:
 - Mean Absolute Error (MAE)
 - Mean Squared Error (MSE)
 - Root Mean Squared Error (RMSE)
 - R^2 Score

9. Interpret the results and comment on the convergence behavior and prediction performance of the model.

Expected Learning Outcomes

Upon successful completion of this experiment, students will be able to:

1. Explain the working of the Gradient Descent optimization algorithm.
2. Implement Linear Regression using Gradient Descent.
3. Analyze the effect of learning rate on model convergence.
4. Evaluate regression models using standard performance metrics.
5. Interpret regression results and assess model performance.

Lab 6

Lab Experiment: Comparison of Logistic Regression and K Nearest Neighbors (KNN) Classifiers

Aim

To implement Logistic Regression and K Nearest Neighbors (KNN) classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset and compare their performance using standard classification evaluation metrics.

Objectives

1. To preprocess the dataset for classification.
2. To implement Logistic Regression and KNN classifiers using Scikit Learn.
3. To evaluate both models using standard performance metrics.
4. To compare the performance of Logistic Regression and KNN and identify the better classifier.

Dataset

Dataset Name: Breast Cancer Wisconsin (Diagnostic)

Source: UCI Machine Learning Repository

Dataset Link:

<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Tasks to Perform

1. Download and load the Breast Cancer Wisconsin (Diagnostic) dataset.
2. Perform exploratory data analysis and data preprocessing.
3. Check for missing values and prepare the dataset for classification.
4. Split the dataset into training and testing sets.
5. Train a Logistic Regression classifier.
6. Train a K Nearest Neighbors (KNN) classifier.
7. Evaluate both classifiers using:
 - Accuracy
 - Precision
 - Recall
 - F1 Score
 - Confusion Matrix
8. Compare the performance of the two classifiers using a comparison table.

9. Interpret the results and comment on the better-performing classifier for the given dataset.

Expected Learning Outcomes

Upon successful completion of this experiment, students will be able to:

1. Implement Logistic Regression and KNN classifiers using Scikit Learn.
2. Apply preprocessing techniques to a real world dataset.
3. Evaluate classification models using standard performance metrics.
4. Compare the performance of multiple classifiers and interpret the results.
5. Recommend the most suitable classifier based on experimental findings.

Lab7

Lab Activity

Dataset: Iris Dataset (`sklearn.datasets.load_iris()`)

Task 1: Dataset Exploration

Load the Iris dataset and answer the following:

1. How many samples and features are present in the dataset?
 2. List the feature names and target classes.
 3. Display the first five records.
 4. Display the class distribution.
-

Task 2: Data Preparation

1. Split the dataset into training (80%) and testing (20%) sets using `random_state=42`.
 2. Briefly explain why the dataset is divided into training and testing sets.
-

Task 3: Building a Decision Tree Classifier

1. Train a Decision Tree classifier using the default parameters.
 2. Predict the class labels for the test dataset.
 3. Evaluate the model using:
 - Accuracy Score
 - Confusion Matrix
 - Classification Report
-

Task 4: Visualizing the Decision Tree

1. Visualize the trained Decision Tree using `plot_tree()`.
 2. From the visualization, identify:
 - Root node
 - Internal nodes
 - Leaf nodes
 - Maximum depth of the tree
 3. Which feature is selected for the first split? Explain why you think this feature was chosen.
-

Task 5: Comparing Gini Index and Entropy

Train two Decision Tree models using the following criteria:

- `criterion='gini'`
- `criterion='entropy'`

Compare the models based on:

1. Accuracy
2. Tree depth
3. Number of leaf nodes
4. Root feature selected
5. Which criterion produces a simpler tree?

Summarize your observations in a table.

Task 6: Effect of Maximum Tree Depth

Train Decision Tree models using the following values of `max_depth`:

- 1
- 2
- 3
- 4
- None

Complete the following table.

Max Depth	Training Accuracy	Testing Accuracy	Tree Depth	Observation
1				
2				
3				
4				
None				

Answer the following:

1. Which model is underfitting?

2. Which model gives the best generalization?
 3. Which model is likely to overfit? Justify your answer.
-

Task 7: Effect of `min_samples_split`

Train Decision Tree models using:

- 2
- 5
- 10
- 20

Complete the table below.

<code>min_samples_split</code>	Accuracy	Tree Depth	Number of Leaf Nodes	Observation
2				
5				
10				
20				

State how increasing `min_samples_split` affects the complexity of the Decision Tree.

Task 8: Effect of `min_samples_leaf`

Train Decision Tree models using:

- 1
- 2
- 5
- 10

Complete the following table.

<code>min_samples_leaf</code>	Accuracy	Tree Depth	Number of Leaf Nodes	Observation
1				
2				

5

10

Explain how changing `min_samples_leaf` influences overfitting.

Task 9: Hyperparameter Tuning

Perform hyperparameter tuning using `GridSearchCV` with the following parameters:

- `criterion`: gini, entropy
- `max_depth`: 2, 3, 4, None
- `min_samples_split`: 2, 5, 10
- `min_samples_leaf`: 1, 2, 4

Report:

1. Best hyperparameter combination.
 2. Best cross validation score.
 3. Test accuracy of the optimized model.
 4. Compare the optimized model with the default Decision Tree model.
-

Task 10: Analysis

Answer the following questions.

1. What is the role of the **criterion** parameter in a Decision Tree?
2. How does `max_depth` influence underfitting and overfitting?
3. Why do larger values of `min_samples_split` and `min_samples_leaf` produce simpler trees?
4. Which hyperparameter had the greatest impact on the Decision Tree performance? Support your answer with observations from the experiments.
5. Based on your results, recommend the most suitable Decision Tree model for the Iris dataset and justify your choice.

Lab 8

Lab 8 : Implementation and Performance Evaluation of Categorical Naive Bayes Classifier

Develop a complete Python program to build a classification pipeline using the standard **Play Tennis dataset**. Your implementation must complete the following sequential tasks:

https://docs.google.com/spreadsheets/d/1mnoUgK8YxInzoDQ7P7iBM1HeZ8tcnUSvMU_H1OWndFA/edit?gid=0#gid=0

1. **Data Preprocessing:**
 - Load the dataset and separate the input features (**Outlook**, **Temperature**, **Humidity**, **Wind**) from the target variable (**Play**).
 - Convert all categorical feature values and target labels into numerical representations using an appropriate encoding technique.
2. **Dataset Partitioning:**
 - Divide the dataset into training and testing subsets using an 80:20 train-test split ratio.
3. **Naive Bayes Model Training & Evaluation:**
 - Train a **Categorical Naive Bayes (CategoricalNB)** model on the training data.
 - Predict the class labels for the test dataset.
 - Calculate and display the overall **Model Accuracy**.
 - Display the **Confusion Matrix** and **Classification Report** (Precision, Recall, F1-Score).
4. **Single-Sample Inference:**
 - Predict whether a person will play tennis under the following specific weather conditions:
 - **Outlook:** Sunny
 - **Temperature:** Cool
 - **Humidity:** High
 - **Wind:** Strong
 - Display both the **predicted class label** and the **corresponding class probabilities**
5. **Model Comparison:**
 - Train a **Decision Tree Classifier** and a **Logistic Regression Classifier**, and an **SVM** on the same training data.
 - Compare all three models in terms of test accuracy, single-sample query prediction, and output class probabilities.

DELIVERABLES

- **Executable Python Script (.py or .ipynb):** Clean, well-commented source code fulfilling all tasks.
- **Output Log:** Console output showing the Naive Bayes evaluation metrics, custom query prediction with probabilities, and the model comparison table.

- **Analysis Report:** A brief explanation (3–4 sentences) comparing why the models produced different predictions and probability scores for the given test instance.

Lab 9

Lab 9

Aim

To implement Support Vector Machine (SVM) for classification and Principal Component Analysis (PCA) for dimensionality reduction, and analyse their effectiveness using real world datasets.

Objectives

1. To implement and evaluate the performance of the Support Vector Machine (SVM) classifier using different kernel functions.
2. To compare the performance of SVM models using standard classification metrics.
3. To apply Principal Component Analysis (PCA) for reducing the dimensionality of a high dimensional dataset.
4. To analyse the variance retained by the principal components and visualize the transformed feature space.
5. To understand the role of supervised learning (SVM) and unsupervised learning (PCA) in machine learning applications.

Part A: Support Vector Machine (SVM)

Aim:

To implement the Support Vector Machine (SVM) classifier for a binary classification problem and evaluate its performance.

Dataset:

UCI Breast Cancer Wisconsin (Diagnostic) Dataset

<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

Tasks:

1. Load the dataset and perform the necessary preprocessing.
2. Split the dataset into training and testing sets (80:20).
3. Train an SVM classifier using a **Linear Kernel** and explore hyper parameter tuning.
4. Evaluate the model using:
 - Accuracy
 - Precision
 - Recall
 - F1 Score

- Confusion Matrix
5. Summarize your observations.
-

Part B: Principal Component Analysis (PCA)

Aim:

To implement Principal Component Analysis (PCA) for dimensionality reduction and analyse the variance retained by the principal components.

Dataset:

UCI Wine Dataset

<https://archive.ics.uci.edu/dataset/109/wine>

Tasks:

1. Load the dataset and perform feature standardization.
2. Apply PCA to reduce the dataset from 13 features to 2 principal components.
3. Display the explained variance ratio of each principal component.
4. Calculate the cumulative explained variance and determine the minimum number of principal components required to retain at least **95%** of the total variance.
5. Visualize the transformed dataset using a two dimensional scatter plot.
6. Compare the original and transformed datasets in terms of:
 - Number of features
 - Information retained
 - Computational efficiency
7. Interpret the significance of the first two principal components.
8. Discuss the advantages, limitations, and applications of PCA in machine learning.

Extra credit Task :

Apply **Linear Discriminant Analysis (LDA)** to the same dataset and reduce it to two linear discriminants. Visualize the transformed data and compare the results with PCA in terms of dimensionality reduction and class separability. Summarize your observations.

Lab 10

Lab Exercise 10: Learning the XOR Boolean Function Using an

MLP

Aim

1. To understand how to implement neural networks using different deep learning libraries (Keras, and TensorFlow).
2. To solve the non-linear XOR problem using an MLP and study the effect of hyperparameters such as learning rate, activation functions, number of neurons, and epochs on model performance.

Question

Implement an MLP to learn the XOR Boolean function

The XOR function takes two binary inputs (0 or 1) and produces a binary output (0 or 1) based on the

following rule:

Input 1	Input 2	XOR Output
---------	---------	------------

0	0	0
---	---	---

0	1	1
---	---	---

1	0	1
---	---	---

1	1	0
---	---	---

Steps:

1. Create the Dataset

- Define input (X) and output (y) arrays for all 4 XOR combinations.

2. Build an MLP

- Input layer: size 2 (for the two inputs).
- Hidden layer: at least 2 neurons with ReLU or Tanh activation.
- Output layer: 1 neuron with sigmoid activation for binary classification.

3. Compile the Model / Define Loss and Optimizer

- Use Binary Cross-Entropy loss.
- Use an optimizer of your choice (e.g., Adam or SGD).

4. Train the Model

- Train the model on the XOR dataset. ○

Experiment with epochs, learning rate, and number of neurons to improve performance.

5. Evaluate the Model

- Predict outputs for all 4 input combinations.
-

Check if the network correctly learns the XOR function.

6. Implement Using Three Libraries

- Repeat the above steps using:

1. Keras (TensorFlow high-level API)

3. TensorFlow low-level API

Additional Exercises (Optional):

- Plot the decision boundary for each implementation.
- Compare training curves and final accuracy between libraries.
- Discuss how changes in learning rate, activation function, hidden layers, or epochs affect learning.

Evaluation Rubrics

Rubrics Marks

Correctness and Demonstration 5 marks

Concept Clarity (Viva) 3 marks

Initiative & Effort (self-learning) 2 marks

Submission Guidelines

1. Make a copy of the lab manual template with your <name

—

reg:no

—

subject name>

2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.

3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation. 4. Create a Git repository in your profile <DL lab-reg no>. Follow a different branch for each lab

<Lab 1, Lab 2 ...> and push the code to Git.

5. Provide the Git link in Google Classroom along with the PDF of the lab manual.

6. Upload the PDF to Google Classroom before the deadline